# Delco

## Manual técnico do projeto

Motor de orquestração de workflows com Python, Django, MySQL, Redis e WebSockets

Copyright (©) 2026 Erik Pimenta

Versão 1.0.0 · Outubro de 2026 · Licença MIT

# Sumário

**Visão geral** 5

- Para que serve 5
- O que o Delco faz 5
- Tecnologias 6
- Identidade visual 6

**Como rodar** 7

- Requisitos 7
- Primeira execução 7
- Comandos do dia a dia 7
- Rodar sem Docker 8

**Acesso e segurança** 9

- Contas de demonstração 9
- Papéis e permissões 9
- Sessões e tokens 9
- Proteções 10

**Escrevendo workflows** 11

- Estrutura de um arquivo 11
- Opções do workflow 11
- Opções da tarefa 12
- O objeto ctx 12
- Passando dados entre tarefas 13
- Tarefas em laço e paralelismo 13
- Idempotência e efeitos externos 13
- Versões dos workflows 13
- Validar antes de publicar 14
- Exemplos incluídos 14

**Guia das telas** 15

- Entrada 15
- Visão geral 15

Workflows 16
Detalhe do workflow 17
Execuções 19
Execução 19
Gaveta da tarefa 20
Workers 22
Minha conta, usuários e tema 22

**Arquitetura** 24

Processos 24
Ciclo de vida de uma tarefa 25
Confiabilidade e retomada 25
Filas e paralelismo 26
Agenda 26
Tempo real 26
Django por dentro 26
Estrutura de pastas 28

**Linha de comando** 29

Comandos de servidor 29
Comandos de cliente 29

**API** 31

Primeiros passos 31
Rotas 31
Erros 32
WebSocket 32

**Configuração** 33

Variáveis 33
Checklist para produção 34

**Testes e automação** 35

Pytest 35
Integração contínua 35

Atividade diária 35

**Solução de problemas** 36

Capítulo 1

# Visão geral

O Delco é um motor de orquestração de workflows de código aberto, no estilo do Apache Airflow e do Temporal. Você descreve um pipeline em Python como um grafo de tarefas com dependências; o Delco agenda as execuções, distribui as tarefas entre workers, repete o que falhar com espera exponencial e mostra tudo em tempo real num painel web.

O projeto foi feito para ser usado, estudado e adaptado. Basta baixar o código, subir com um comando e escrever seus próprios arquivos de workflow na pasta `workflows/`. Nada de dados reais ou segredos vem no repositório.

## Para que serve

Qualquer rotina com várias etapas que precisa rodar sozinha, no horário certo, sem perder o fio quando algo dá errado: ETL e relatórios, integrações com fornecedores instáveis, sincronizações, backups, migrações em lotes, processamento de arquivos e notificações.

O diferencial de engenharia é a **confiabilidade**: o estado de cada execução vive no MySQL. Se o servidor cair no passo 23 de um fluxo de 50, ao voltar o Delco sabe exatamente onde parou e refaz só o passo 23.

## O que o Delco faz

### Para quem escreve workflows

- Workflows como código Python, com dependências, parâmetros, agenda cron, fila, tempo limite e retentativas por tarefa.
- Resultado de cada tarefa disponível para as seguintes em `ctx.upstream`.
- Regras de disparo (`all_success`, `all_done`, `one_failed`) para limpezas e alertas.
- Tarefas geradas em laço, funções `async`, progresso (0 a 100%) e chave de idempotência pronta.
- Validação dos arquivos sem rodar nada: `delco validate`.

### Para quem opera

- Painel com grafo ao vivo, grade de execuções, linha do tempo e logs que chegam enquanto a tarefa roda.
- Disparo manual com parâmetros, cancelamento, retomada a partir da falha ou de qualquer tarefa, pausa da agenda.
- Quadro de vagas dos workers e situação de cada fila.
- CLI e API REST com as mesmas ações do painel.

### Para quem administra

- Três papéis (Leitura, Operador, Administrador), tokens pessoais com validade e auditoria de todas as ações.
- Vários workers e agendadores ao mesmo tempo, sem execução em dobro.
- Limpeza automática do histórico antigo e configuração por variáveis de ambiente.

## Tecnologias

| Camada | Tecnologia | Papel |
|---|---|---|
| Linguagem | Python 3.13 | SDK, agendador, workers, API e CLI |
| Framework | Django 6.1 | Modelos, migrações, comandos de gerenciamento, configuração e segurança |
| API | Django REST Framework + drf-spectacular | REST com permissões por papel e documentação OpenAPI/Swagger |
| Servidor | Daphne (ASGI) | HTTP e WebSocket no mesmo processo |
| Banco | MySQL 8.4 com o ORM do Django | Fonte da verdade de todo o estado (mysqlclient, sessão em UTC) |
| Fila | Redis 7 Streams | Entrega das tarefas aos workers com grupo de consumidores |
| Tempo real | Django Channels + Redis pub/sub | Eventos de execução, tarefa, worker e log pelo WebSocket |
| Painel | HTML, CSS e JavaScript (sem build) | Interface servida pelo próprio Django com WhiteNoise |
| Agenda | croniter | Expressões cron com fuso horário |
| Testes | Pytest + pytest-cov | 77 testes com MySQL e Redis reais |
| Execução | Docker Compose | MySQL, Redis, API, agendador e workers |

## Identidade visual

A marca é a grade de execuções do próprio painel: um quadro 3 × 3 com seis casas cheias (tarefas concluídas) e três vazadas (tarefas por fazer). Ela é sempre preta e branca. A interface usa uma paleta de tons terrosos e frios: fundo linho, ardósia como cor de destaque e de tarefa rodando, sálvia para sucesso e areia para o que ainda está pendente, com a fonte Public Sans. Falha e nova tentativa usam um tijolo e um ocre suaves, tirados dos mesmos tons, para continuarem fáceis de distinguir.

| **Oliva** #65695B | **Sálvia** #9CA38E | **Névoa** #C0C4B7 | **Aço** #8A9EAB | **Ardósia** #4C6170 | **Linho** #EBE0DD | **Areia** #D9C3B6 |
|---|---|---|---|---|---|---|

Capítulo 2

# Como rodar

## Requisitos

- **Docker** com Docker Compose (Docker Desktop no Windows e no macOS).
- Cerca de 1 GB livre para as imagens e a porta 8080 livre (dá para trocar no `.env`).
- Para rodar sem Docker: Python 3.12 ou mais novo, MySQL 8.4+ e Redis 6.2+ (no Linux e no macOS, o mysqlclient pede a biblioteca cliente do MySQL: no Ubuntu, `sudo apt install pkg-config libmysqlclient-dev`).

## Primeira execução

1. Baixe o repositório (**Code → Download ZIP** no GitHub ou `git clone`).
2. **Windows:** dê dois cliques em `iniciar.cmd`. **Linux ou macOS:** rode `./iniciar.sh`.
3. Na primeira vez o script copia o `.env.example` para `.env` e gera uma senha aleatória para o MySQL.
4. O Docker monta a imagem e sobe 6 contêineres: mysql, redis, api, scheduler e dois workers.
5. Quando a API responde, o navegador abre em `http://localhost:8080`. Entre com uma conta de demonstração.

> A primeira subida baixa as imagens e pode levar alguns minutos. As seguintes sobem em segundos.

## Comandos do dia a dia

| Ação | Comando |
|---|---|
| Ver os contêineres | `docker compose ps` |
| Acompanhar os logs | `docker compose logs -f scheduler worker` |
| Mais workers | `docker compose up -d --scale worker=4` |
| Parar sem perder dados | `docker compose stop` |
| Subir de novo | `docker compose up -d` |
| Atualizar depois de mudar o código | `docker compose up -d --build` |
| Apagar tudo, inclusive o banco | `docker compose down -v` |
| Abrir um terminal no Delco | `docker compose exec api bash` |

Arquivos novos ou alterados na pasta `workflows/` são lidos pelo agendador em até 30 segundos, sem reiniciar nada: a pasta é montada dentro dos contêineres do agendador e dos workers.

## Rodar sem Docker

Útil para desenvolver workflows com o depurador do seu editor.

```
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt && pip install -e .
export DELCO_DATABASE_URL=mysql://delco:SENHA@localhost:3306/delco
export DELCO_REDIS_URL=redis://localhost:6379/0
python manage.py migrate && python manage.py seed   # tabelas, contas e histórico de demonstração
python manage.py runserver 8080      # terminal 1: API, WebSocket e painel (Daphne)
python manage.py scheduler           # terminal 2: agendador
python manage.py worker --concurrency 4   # terminal 3 (e quantos mais quiser)
```

Os mesmos processos têm atalhos mais curtos: `delco migrate`, `delco seed`, `delco api --port 8080`, `delco scheduler` e `delco worker`.

Capítulo 3

# Acesso e segurança

## Contas de demonstração

Todas usam a senha `Delco@2026`. São fictícias (domínio `delco.dev`) e existem só para testes. Elas são criadas quando `DELCO_SEED=true` e o banco não tem nenhum usuário.

| Papel | E-mail | Nome |
|---|---|---|
| Administrador | `admin@delco.dev` | Helena Prado |
| Operador | `operador@delco.dev` | Tiago Moura |
| Leitura | `leitura@delco.dev` | Bruna Sales |

> Antes de usar de verdade: suba com `DELCO_SEED=false` num banco vazio e crie o primeiro administrador com `python manage.py createuser --role admin --email voce@empresa.com --name "Seu Nome"` (ou `delco users create`). Se já subiu com as contas de demonstração, troque as senhas ou desative-as na tela Usuários.

## Papéis e permissões

| Ação | Leitura | Operador | Administrador |
|---|---|---|---|
| Ver workflows, execuções, logs, workers e filas | sim | sim | sim |
| Criar e revogar os próprios tokens, trocar a própria senha | sim | sim | sim |
| Disparar, cancelar e retomar execuções | não | sim | sim |
| Pausar e retomar a agenda de um workflow | não | sim | sim |
| Criar usuários, trocar papéis, desativar e desbloquear contas | não | não | sim |
| Consultar a auditoria | não | não | sim |

## Sessões e tokens

O login devolve um **token de sessão** opaco, válido por 12 horas (`DELCO_SESSION_HOURS`). O painel o guarda no `sessionStorage` da aba: fechar a aba encerra o acesso. Para a CLI, scripts e integrações, crie um **token pessoal** em Minha conta, com validade de 30 dias a sem validade. O token aparece uma única vez; no banco fica apenas o SHA-256 dele.

Todo pedido à API leva `Authorization: Bearer <token>`. O WebSocket recebe o token em `/api/ws?token=…`.

## Proteções

| Proteção | Como funciona |
|---|---|
| Senhas | Hasher scrypt nativo do Django (`ScryptPasswordHasher`), com sal aleatório. Mínimo de 8 caracteres com letras e números. |
| Bloqueio | 5 senhas erradas seguidas bloqueiam a conta por 5 minutos. E-mail inexistente gasta o mesmo tempo de verificação. |
| Troca de senha | Encerra as outras sessões do usuário. Desativar uma conta revoga todas as sessões e tokens. |
| Isolamento | Cada tentativa roda num processo filho com grupo próprio; ao cancelar ou estourar o tempo, o grupo inteiro é encerrado. |
| Cabeçalhos | Content-Security-Policy nativa do Django 6 restrita ao próprio domínio, X-Frame-Options DENY, nosniff e no-store na API. |
| Hosts | `ALLOWED_HOSTS` do Django vem de `DELCO_ALLOWED_HOSTS`; o WebSocket confere a origem com o mesmo valor. |
| Auditoria | Login, falha de login, disparo, cancelamento, retomada, pausa, tokens e usuários ficam em audit_log. |
| Segredos | Somente no arquivo .env, que está no .gitignore. O repositório não tem dados pessoais reais. |

Capítulo 4

# Escrevendo workflows

## Estrutura de um arquivo

Cada arquivo `.py` da pasta de workflows pode definir um ou mais objetos `Workflow`. Arquivos que começam com `_` são ignorados (use para módulos auxiliares).

```
from delco import Workflow

vendas = Workflow(
    "relatorio_vendas",
    description="Extrai, agrega e publica o relatório do dia.",
    schedule="0 7 * * *",
    timezone="America/Sao_Paulo",
    params={"dias": 1},
    max_active_runs=1,
    tags=["etl"],
)

@vendas.task(retries=3, retry_delay=10, timeout=120)
def extrair(ctx):
    ctx.log(f"Lendo {ctx.params['dias']} dia(s)")
    return {"linhas": 1200}

@vendas.task(depends_on=[extrair])
def publicar(ctx):
    linhas = ctx.upstream["extrair"]["linhas"]
    ctx.progress(100, f"{linhas} linhas publicadas")
```

## Opções do workflow

| Parâmetro | Padrão | Descrição |
|---|---|---|
| `name` | obrigatório | Letras minúsculas, números e _ (até 63). É a chave do workflow no banco. |
| `description` | vazio | Texto mostrado no painel. |
| `schedule` | `None` | Cron de 5 campos ou `@hourly`, `@daily`, `@weekly`, `@monthly`. Sem agenda = só manual. |
| `timezone` | America/Sao_Paulo | Fuso usado para interpretar a agenda. |
| `max_active_runs` | 1 | Execuções simultâneas do mesmo workflow; as demais esperam na fila. |
| `params` | `{}` | Parâmetros padrão; o disparo manual pode sobrescrever qualquer chave. |
| `tags` | `()` | Etiquetas para busca no painel. |

| Parâmetro | Padrão | Descrição |
|---|---|---|
| `default_retries`, `default_retry_delay`, `default_timeout`, `default_queue` | 2, 5 s, 300 s, default | Padrões para as tarefas que não informarem esses valores. |

## Opções da tarefa

| Parâmetro | Padrão | Descrição |
|---|---|---|
| `depends_on` | `[]` | Funções decoradas ou nomes das tarefas que precisam terminar antes. |
| `retries` | 2 | Novas tentativas depois da primeira falha (0 a 100). |
| `retry_delay` | 5 s | Espera antes da segunda tentativa. |
| `backoff` | 2 | Multiplicador da espera: `retry_delay × backoff^(tentativa−1)`, com ±10% de variação. |
| `max_retry_delay` | 600 s | Teto da espera entre tentativas. |
| `timeout` | 300 s | Tempo máximo de cada tentativa. `None` = sem limite. |
| `queue` | default | Fila em que a tarefa é publicada. Workers escolhem quais filas consomem. |
| `trigger_rule` | all_success | `all_success`: todas as dependências ok. `all_done`: roda sempre que todas terminarem. `one_failed`: só se alguma falhar. |
| `name`, `description` | nome e docstring da função | Como a tarefa aparece no painel. |

## O objeto ctx

Toda tarefa pode receber um argumento `ctx` (se a função não tiver parâmetros, o Delco a chama sem argumentos).

| Atributo ou método | O que é |
|---|---|
| `ctx.params` | Parâmetros da execução: os padrões do workflow mais o que veio no disparo. |
| `ctx.upstream` | Dicionário com o resultado de cada dependência direta, pelo nome. |
| `ctx.run_id`, `ctx.task_name`, `ctx.workflow` | Identificação da execução e da tarefa. |
| `ctx.attempt`, `ctx.max_attempts` | Número desta tentativa e o limite. `ctx.is_retry` e `ctx.is_last_attempt` ajudam a decidir. |
| `ctx.logical_date` | Horário agendado que originou a execução (ou `None` no disparo manual). |
| `ctx.idempotency_key` | Igual em todas as tentativas da mesma tarefa na mesma execução. Use para não repetir efeitos externos. |

| Atributo ou método | O que é |
|---|---|
| `ctx.log(msg, level)` | Linha no log da tentativa ( `info` , `warning` , `error` , `debug` ). `print` e `logging` também são capturados. |
| `ctx.progress(pct, msg)` | Andamento de 0 a 100, mostrado como barra no grafo e no quadro de workers. |

## Passando dados entre tarefas

O valor devolvido por uma tarefa é convertido para JSON e gravado no banco ( `task_instances.output` ). As tarefas seguintes o recebem em `ctx.upstream` . O limite é de 1 MB ( `DELCO_MAX_OUTPUT_BYTES` ): para volumes maiores, grave em arquivo, banco ou armazenamento e devolva só o caminho, como faz o exemplo `relatorio_vendas_diario` . Datas e decimais são convertidos para texto.

Como o resultado fica gravado, uma tarefa que já terminou nunca precisa rodar de novo para alimentar as seguintes: é isso que permite retomar a execução do ponto em que parou.

## Tarefas em laço e paralelismo

Tarefas sem dependência entre si rodam em paralelo, em quantas vagas houver. Para gerar um leque de tarefas, use `add_task` num laço (o parâmetro padrão `regiao=_r` fixa o valor de cada volta):

```
for _r in ["norte", "sul", "sudeste"]:
    def agregar(ctx, regiao=_r):
        return {"regiao": regiao, "total": calcular(regiao)}
    vendas.add_task(f"agregar_{_r}", agregar, depends_on=[extrair])

@vendas.task(depends_on=["agregar_norte", "agregar_sul", "agregar_sudeste"])
def consolidar(ctx):
    return sum(v["total"] for v in ctx.upstream.values())
```

## Idempotência e efeitos externos

Uma tarefa pode rodar mais de uma vez: numa retentativa, depois de uma queda de worker ou numa retomada manual. Escreva tarefas que possam ser repetidas sem estrago:

- Use `ctx.idempotency_key` como chave de deduplicação em APIs (cabeçalho `Idempotency-Key` ) ou como chave única num `INSERT ... ON CONFLICT DO NOTHING` .
- Prefira sobrescrever arquivos (mesmo nome) a acrescentar ao final.
- Os utilitários `delco.helpers.http_json` e `delco.helpers.shell` já enviam a chave e jogam a saída no log.

## Versões dos workflows

Ao ler a pasta, o agendador calcula uma impressão digital (SHA-256) do grafo de cada workflow: tarefas, dependências e opções. Mudou o grafo, nasce uma nova versão em `workflow_versions` . Execuções em andamento continuam com o grafo da versão em que começaram; as novas usam a atual. Mudar só o corpo de uma função não cria versão: o worker sempre importa o arquivo atual a cada tentativa.

> Se você remover ou renomear uma tarefa enquanto uma execução antiga ainda vai rodá-la, a tentativa falha com a mensagem "A tarefa não existe mais". Retome a execução depois de ajustar o arquivo.

## Validar antes de publicar

`delco validate workflows` importa cada arquivo, confere nomes, dependências, ciclos e agendas e mostra uma tabela com tarefas e versão. Termina com código 1 se algum arquivo tiver erro, então serve também como passo de CI. No painel, arquivos com erro aparecem num aviso no topo da tela Workflows; os demais continuam funcionando normalmente.

## Exemplos incluídos

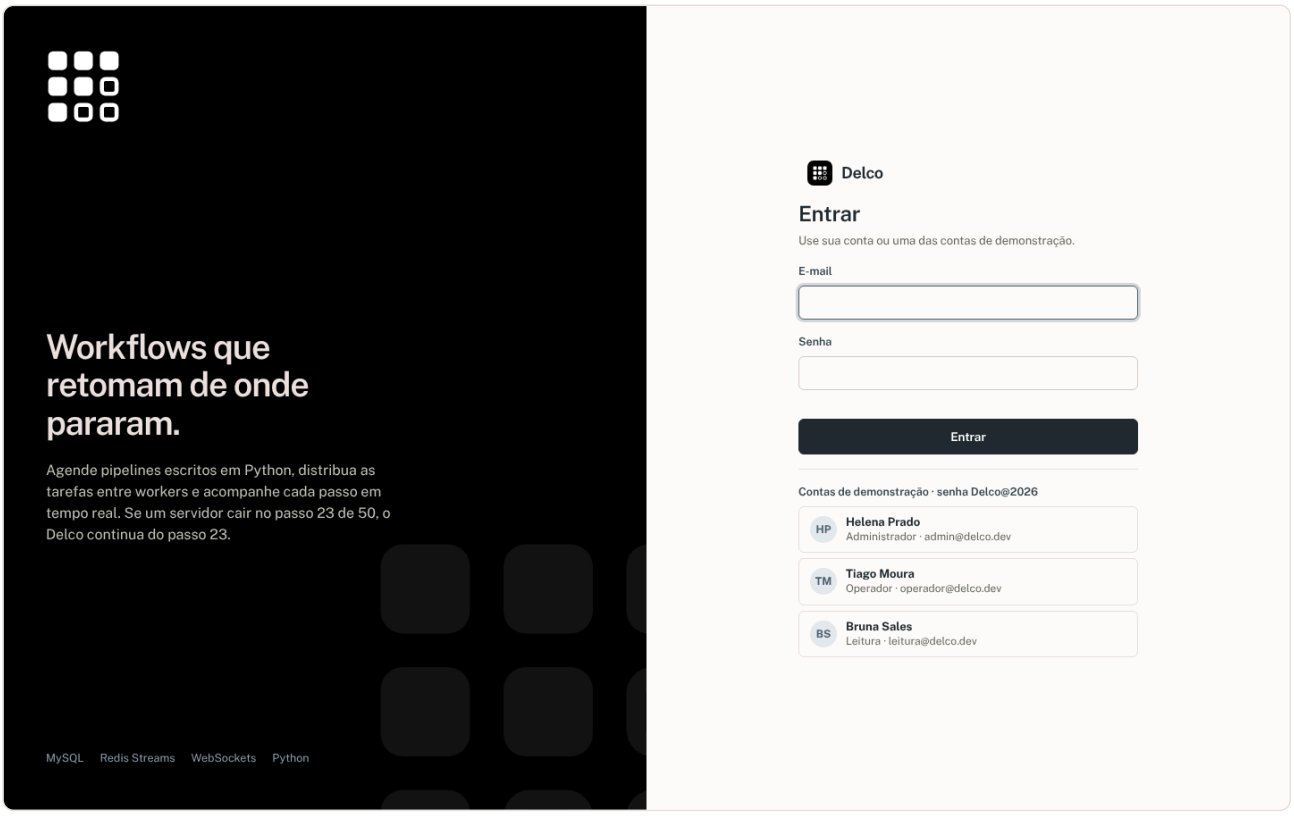
| Workflow | Agenda | O que demonstra |
|---|---|---|
| `ola_delco` | manual | O mínimo: duas tarefas e passagem de resultado. |
| `relatorio_vendas_diario` | todo dia 07:00 | ETL com leque paralelo de 5 regiões, arquivos intermediários e CSV final. |
| `sincronizar_catalogo` | a cada 15 min | Fornecedor instável (falhas simuladas), retentativas com espera, lotes em paralelo, all_done e one_failed. |
| `backup_noturno` | todo dia 02:30 | Fila `critical`, arquivos reais, gzip, SHA-256 e retenção. |
| `migracao_em_50_passos` | manual | 50 passos em sequência para testar a retomada derrubando um worker. |

Capítulo 5

# Guia das telas

## Entrada

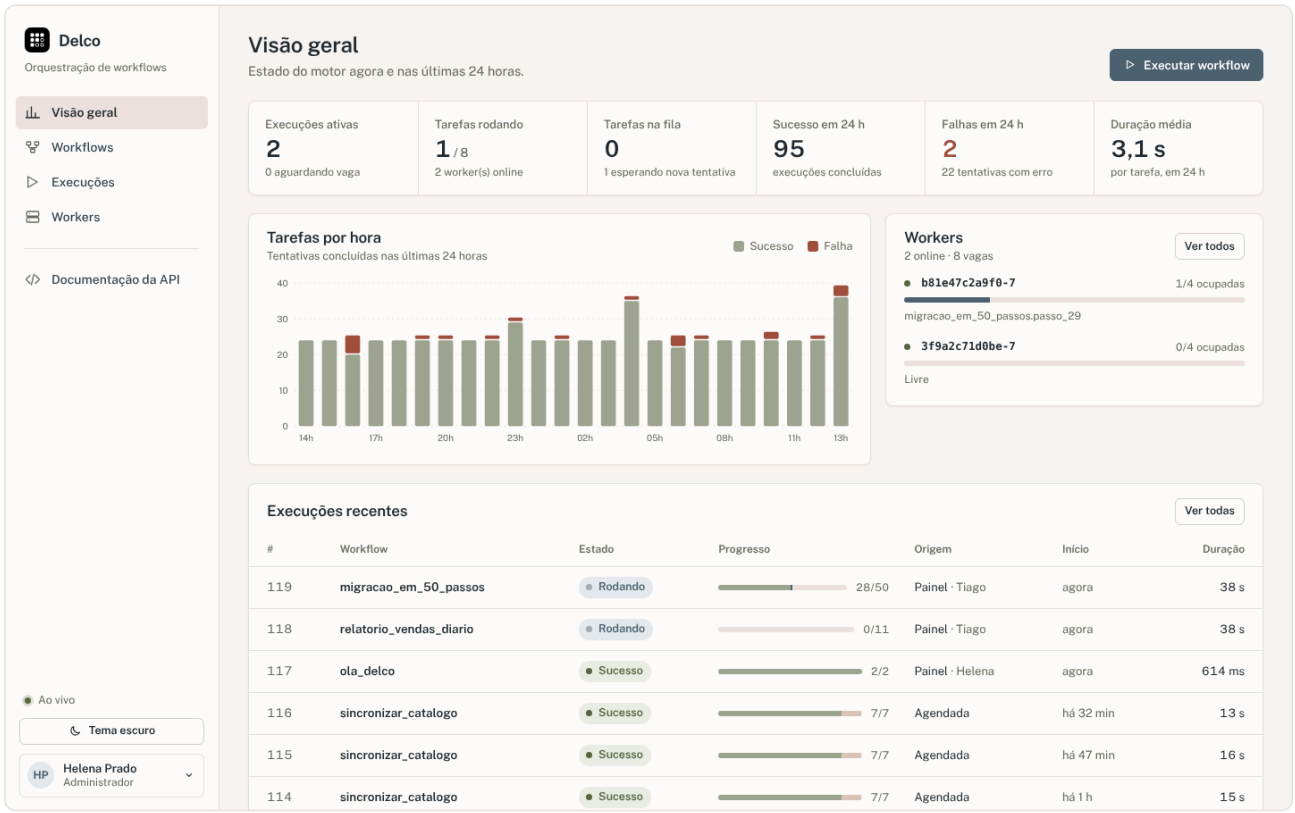
E-mail e senha, com atalho para as três contas de demonstração. O painel lembra o tema claro ou escuro escolhido no navegador.




Tela de entrada

## Visão geral

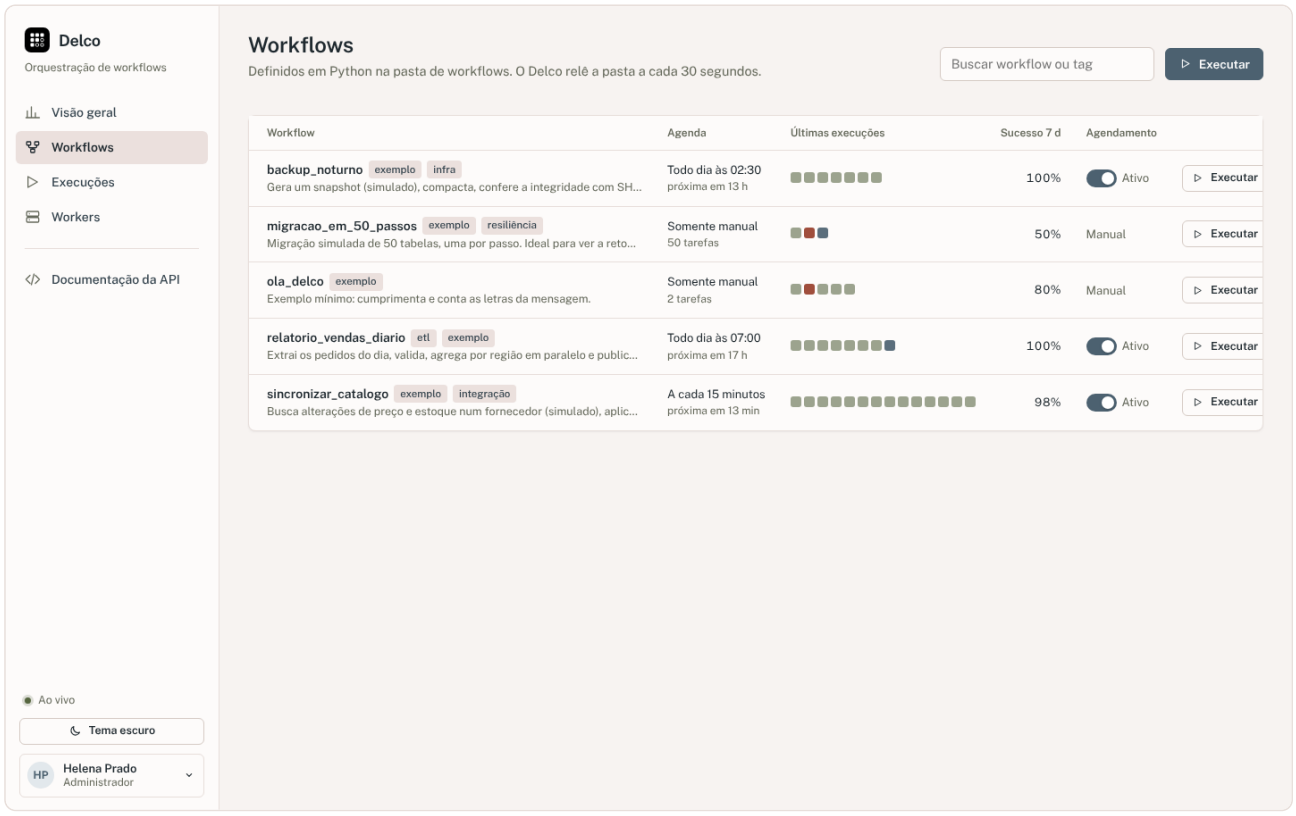
Indicadores do momento (execuções ativas, tarefas rodando sobre o total de vagas, fila, sucessos e falhas em 24 horas, duração média), o gráfico de tentativas por hora, a ocupação de cada worker e as execuções recentes. Tudo se atualiza sozinho pelo WebSocket; o ponto Ao vivo no menu indica a conexão.




Visão geral

## Workflows

Lista com descrição, etiquetas, agenda em português e a próxima execução, os quadradinhos das últimas execuções (clique para abrir), a taxa de sucesso em 7 dias, a chave para pausar a agenda e o botão Executar, que abre o diálogo de parâmetros em JSON.

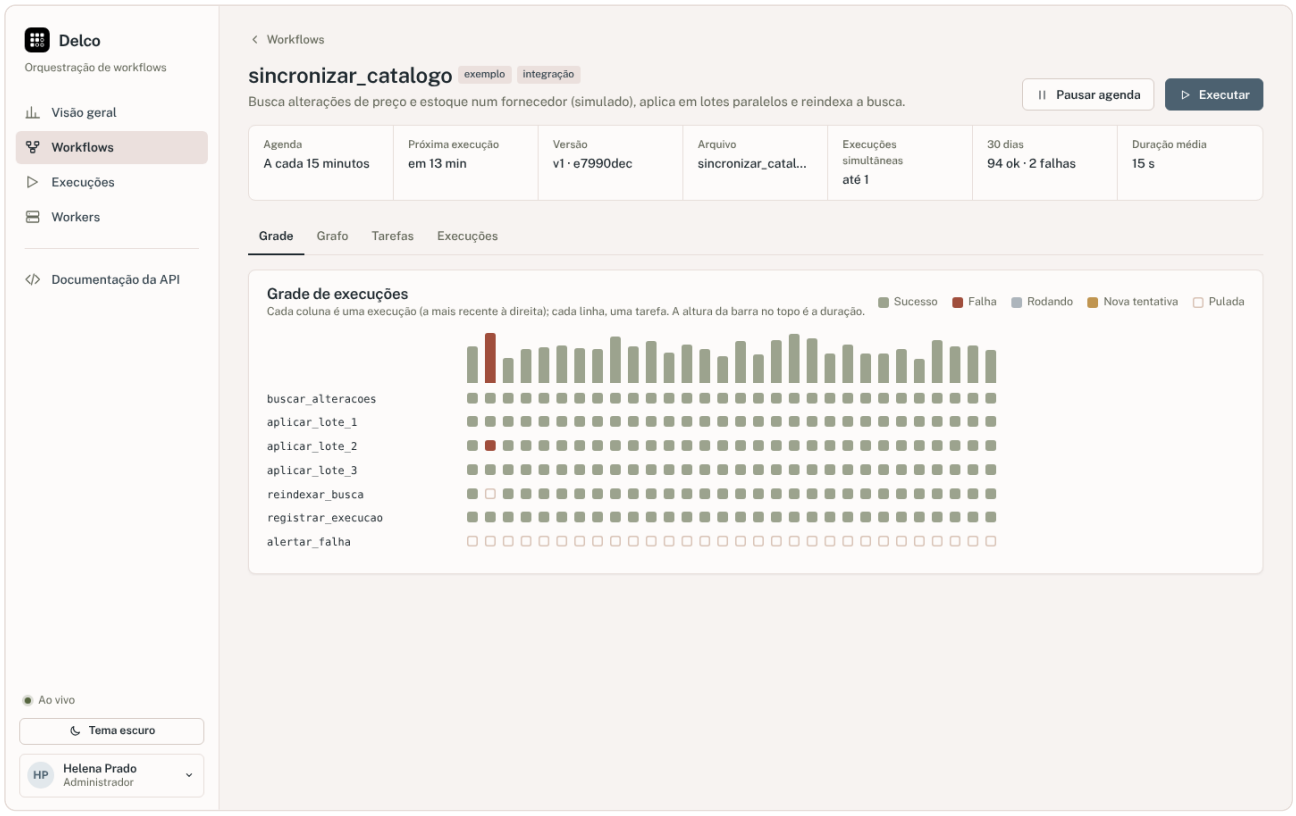



Lista de workflows

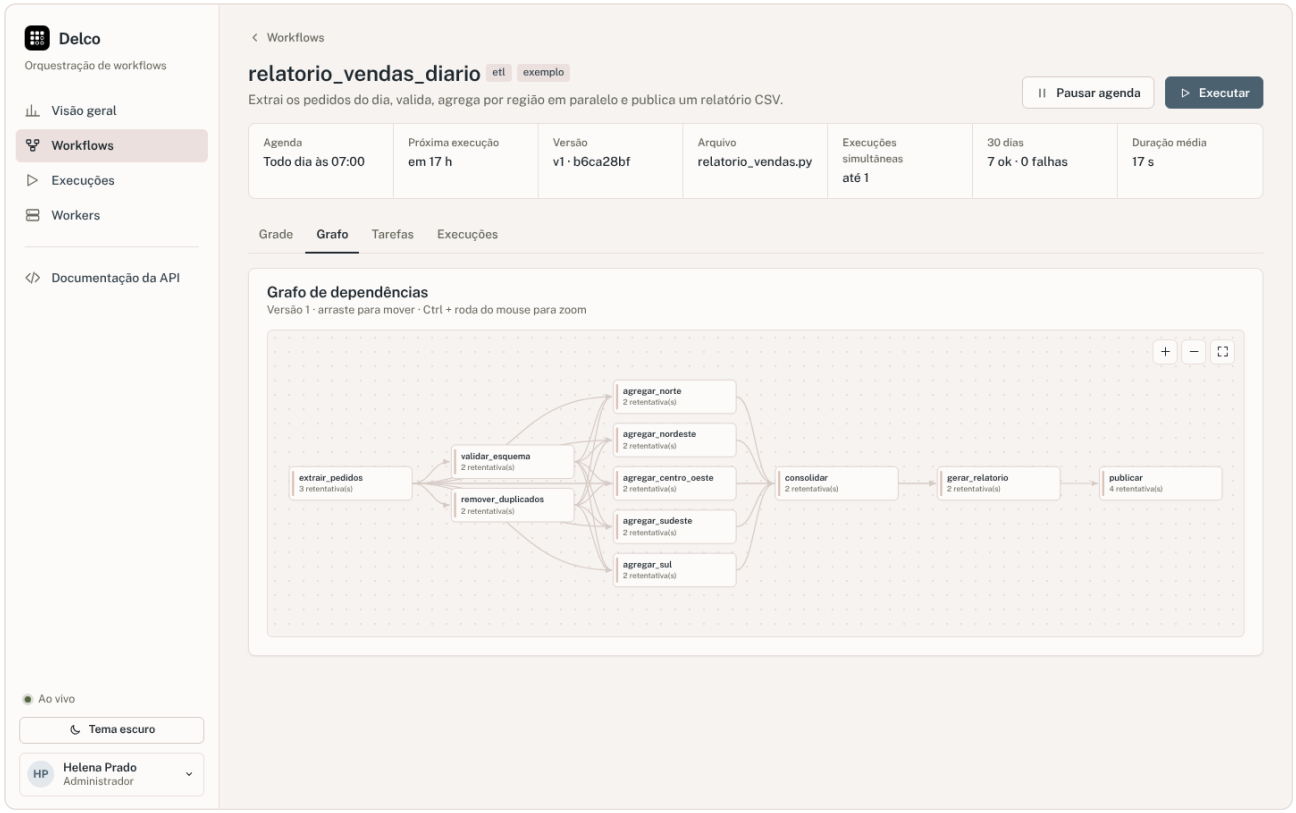
## Detalhe do workflow

Faixa com agenda, próxima execução, versão, arquivo, limite de execuções simultâneas e números dos últimos 30 dias. Quatro abas:

- **Grade:** cada coluna é uma execução, cada linha uma tarefa. A barra no topo mostra a duração. Ótima para ver qual passo falha com frequência.
- **Grafo:** as dependências da versão atual. Arraste para mover e use Ctrl + roda do mouse ou os botões para zoom.
- **Tarefas:** as opções de cada tarefa (retentativas, espera, tempo limite, fila e regra).
- **Execuções:** o histórico só deste workflow.



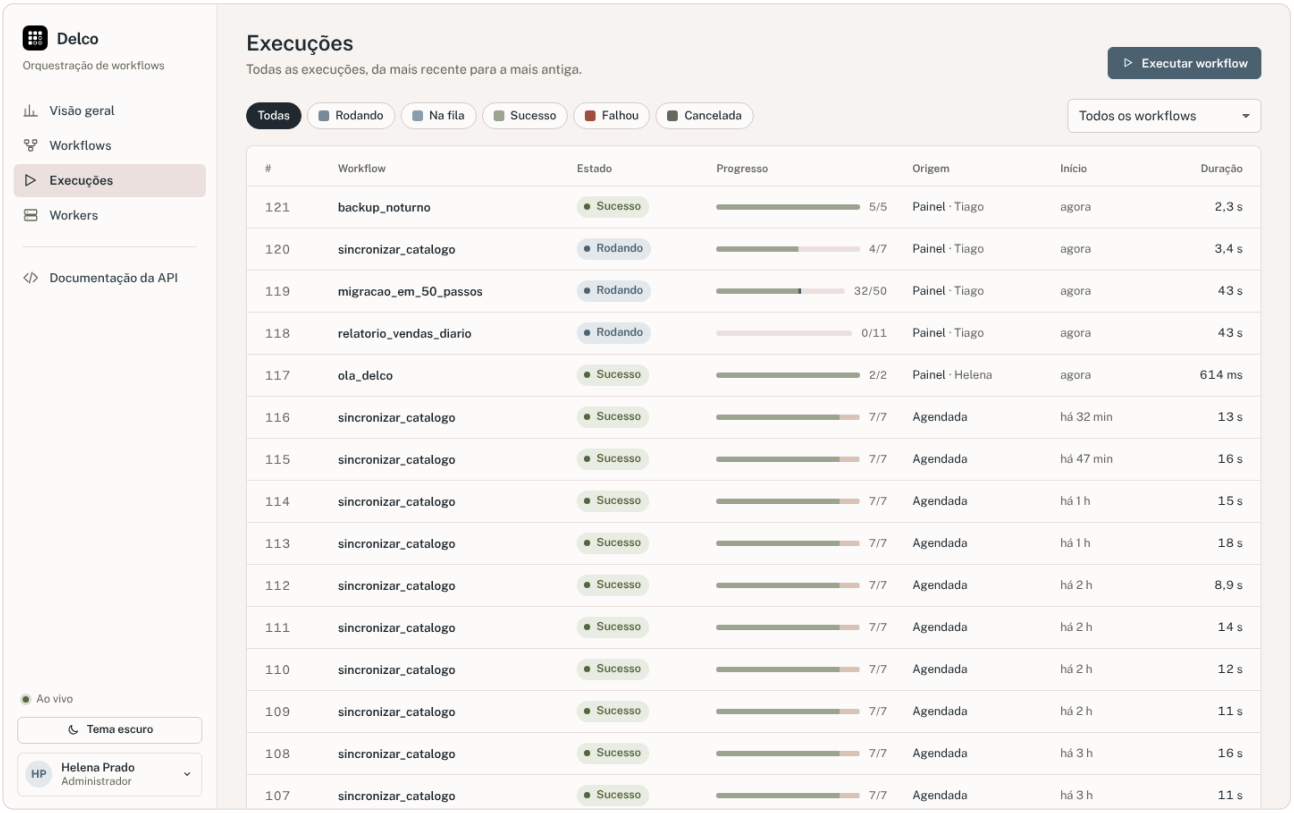

Aba Grade de um workflow agendado a cada 15 minutos




Aba Grafo: leque paralelo de agregações por região

## Execuções

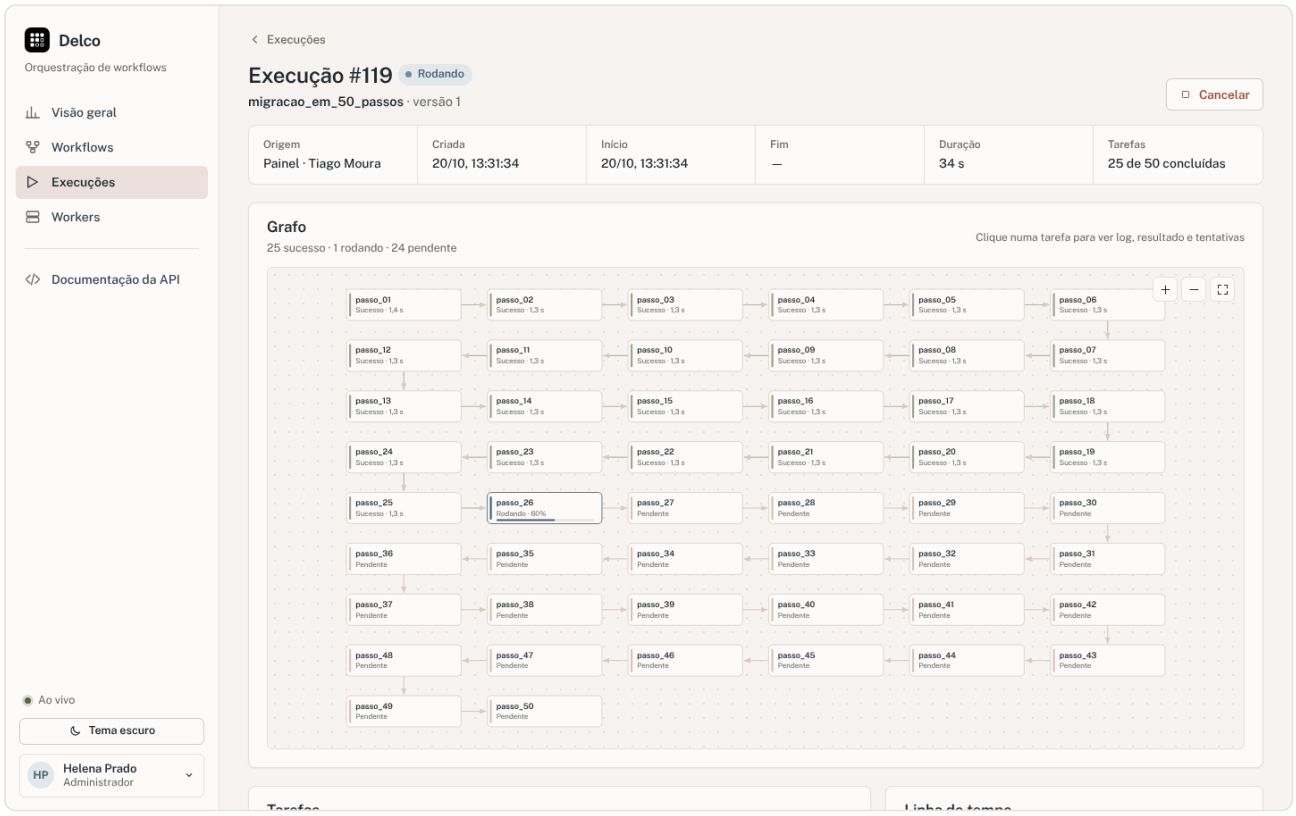
Todas as execuções com filtros por estado e por workflow, paginação e barra de progresso (sálvia: sucesso, tijolo: falha, ardósia: rodando, contorno vazio: pulada).




Lista de execuções

## Execução

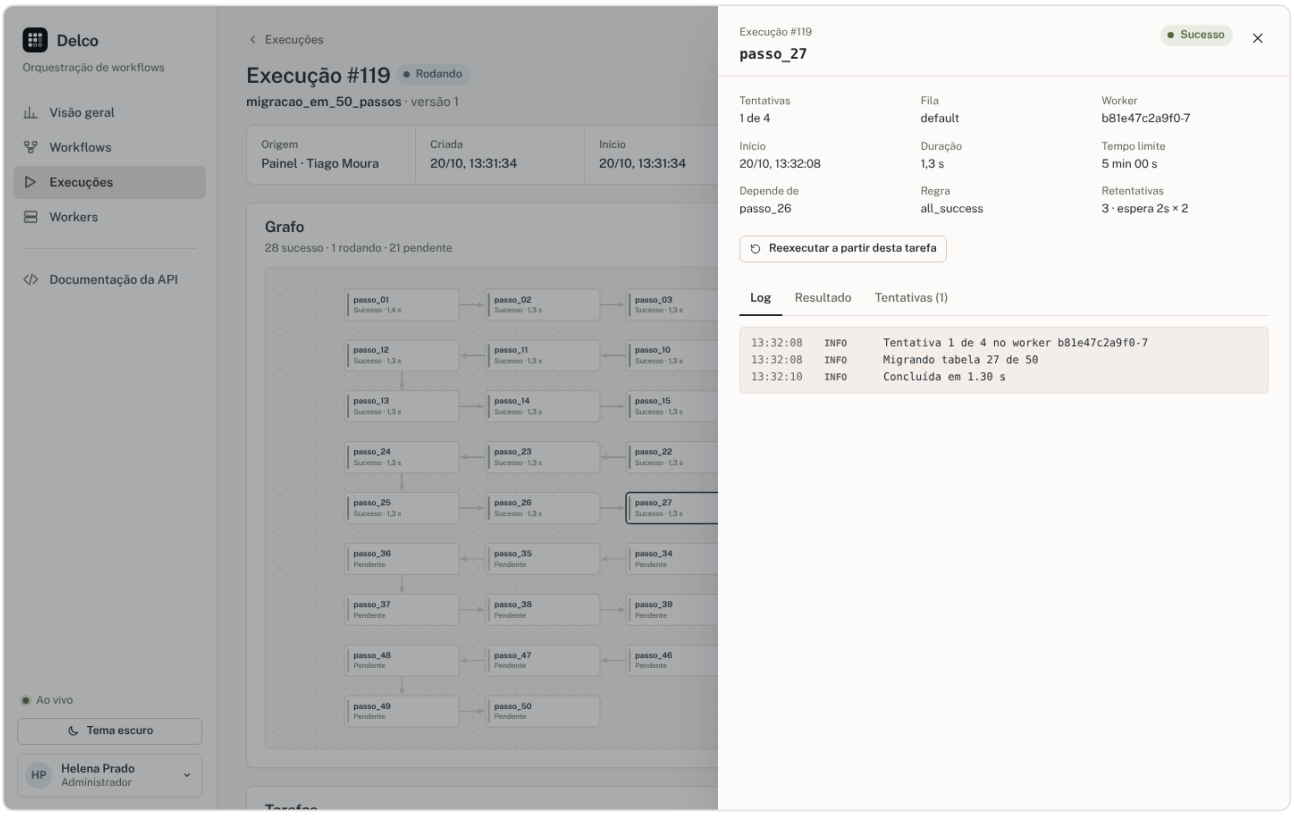
Cabeçalho com estado, origem (painel, CLI, API ou agenda), horários e duração. O **grafo ao vivo** pinta cada tarefa pelo estado e mostra o progresso das que estão rodando; fluxos longos em sequência são quebrados em zigue-zague. Abaixo ficam a tabela de tarefas, a **linha do tempo** e os parâmetros usados. As ações são **Cancelar**, **Retomar da falha** (refaz só o que falhou, foi pulado ou cancelado) e **Executar de novo**.



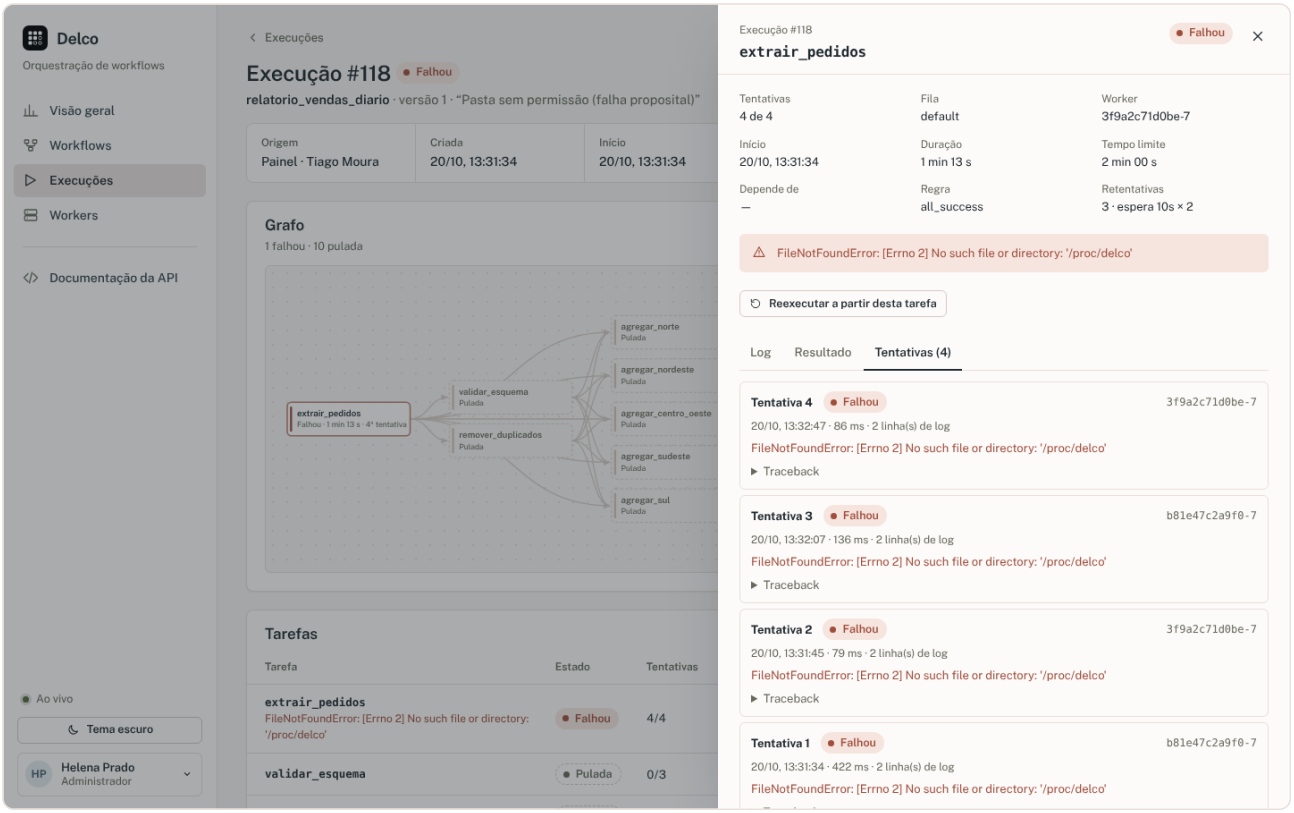

Execução de 50 passos em andamento

## Gaveta da tarefa

Clique numa tarefa (no grafo, na tabela ou na linha do tempo) para abrir a gaveta: tentativas, fila, worker, horários, regra e política de espera; o **log** chega linha a linha enquanto a tarefa roda; a aba **Resultado** mostra o JSON devolvido e **Tentativas** lista cada tentativa com worker, duração, erro e traceback. O botão Reexecutar a partir desta tarefa refaz a tarefa e tudo o que depende dela.



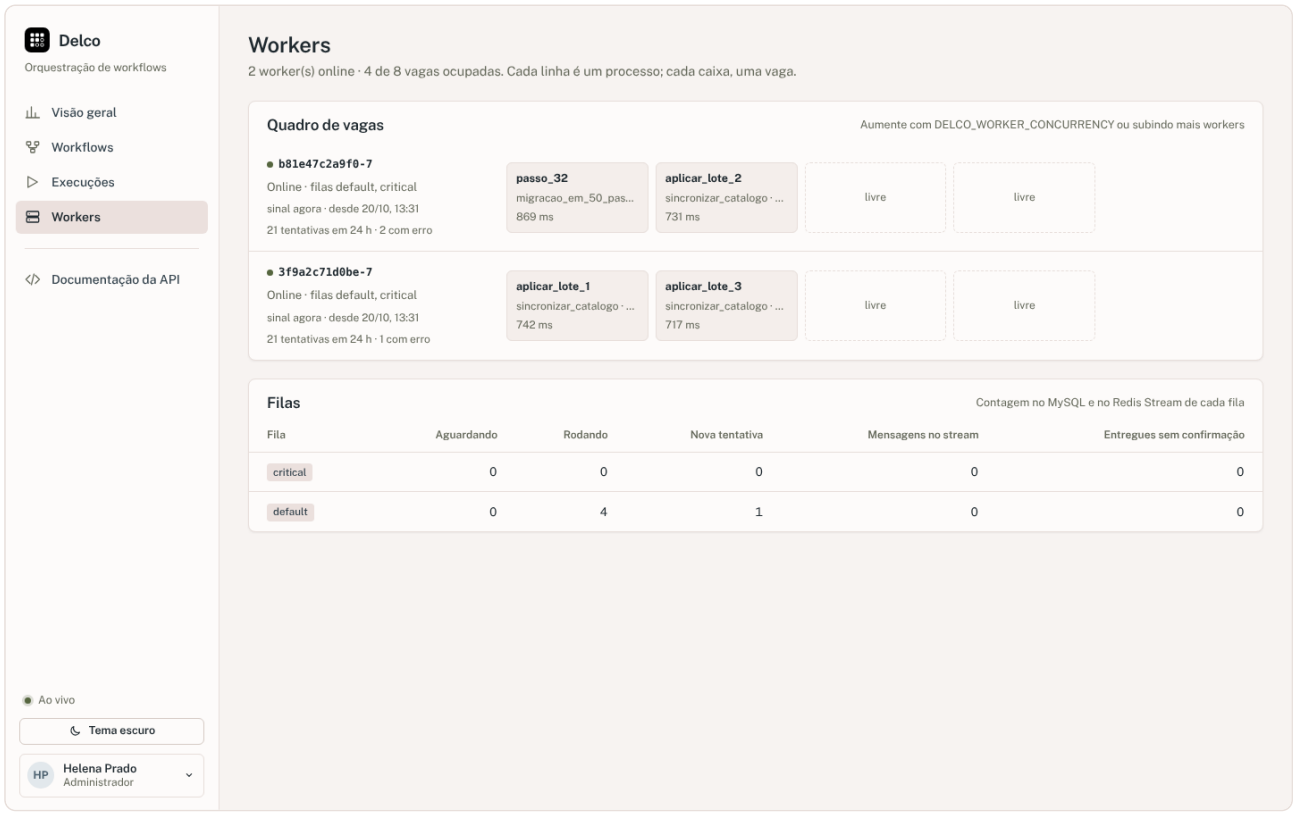

Gaveta com o log de uma tarefa




Execução com falha: as quatro tentativas registradas e as tarefas seguintes puladas

## Workers

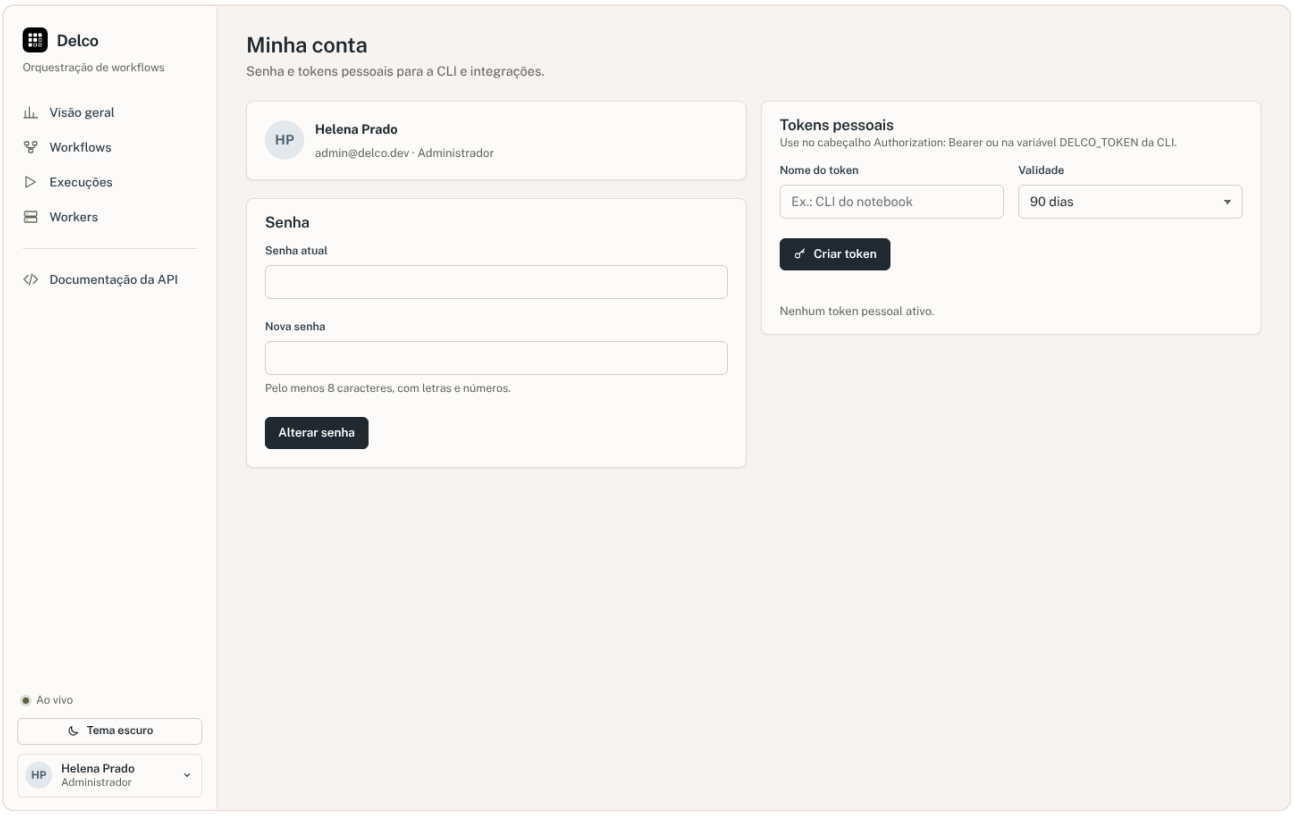
O **quadro de vagas** mostra cada worker online, as filas que consome, o último sinal e cada vaga: ocupada (tarefa, execução, tempo e progresso) ou livre. Workers desligados na última hora aparecem numa linha abaixo. A tabela **Filas** compara o MySQL (aguardando, rodando, nova tentativa) com o Redis Stream (mensagens e entregues sem confirmação).



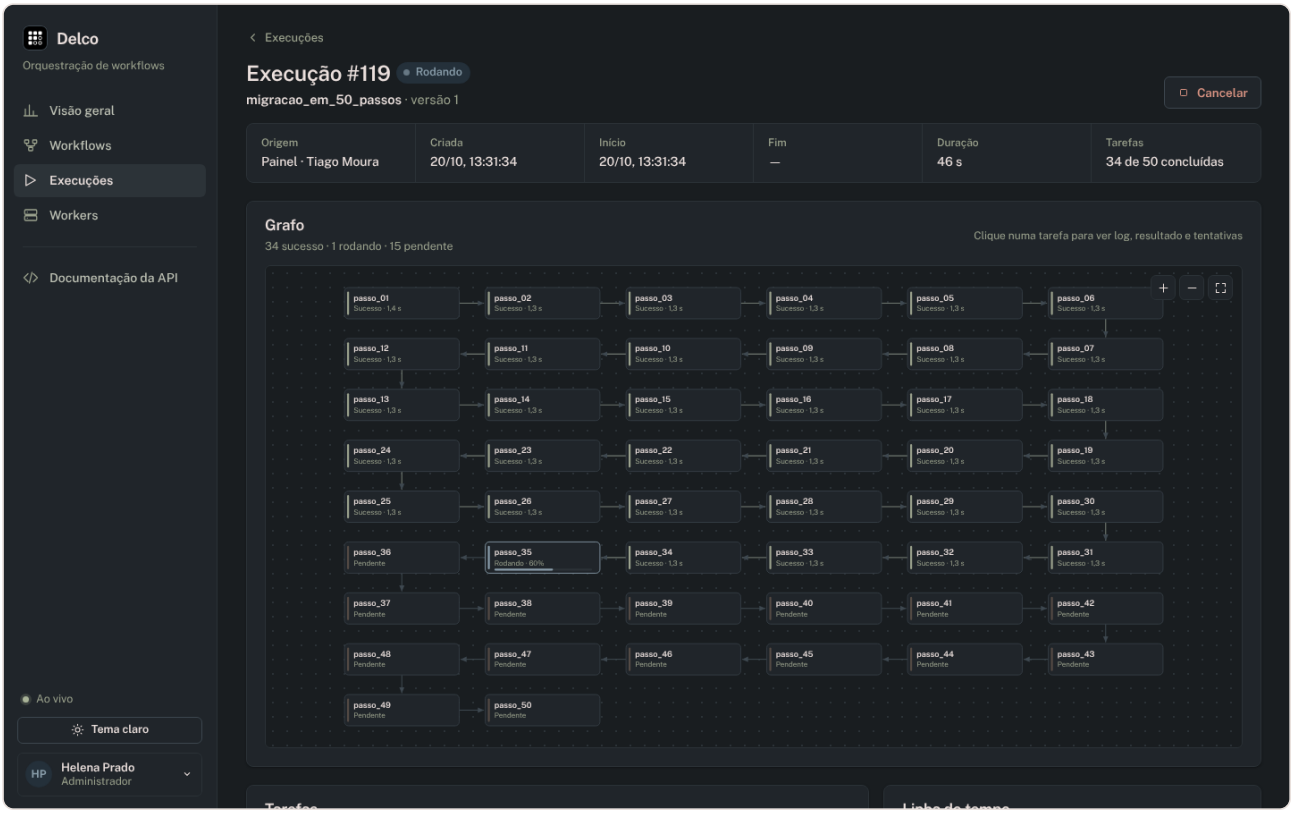

Quadro de vagas e filas

## Minha conta, usuários e tema

Minha conta troca a senha e gerencia tokens pessoais. Usuários (administrador) cria contas, troca papéis, desativa e desbloqueia, e mostra as últimas ações da auditoria. O botão de tema alterna entre claro e escuro.



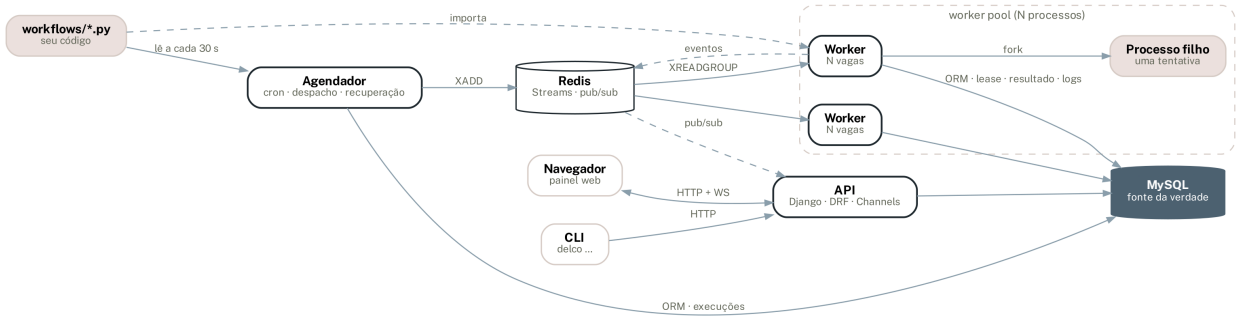

Minha conta e tokens pessoais




Tema escuro

Capítulo 6

# Arquitetura

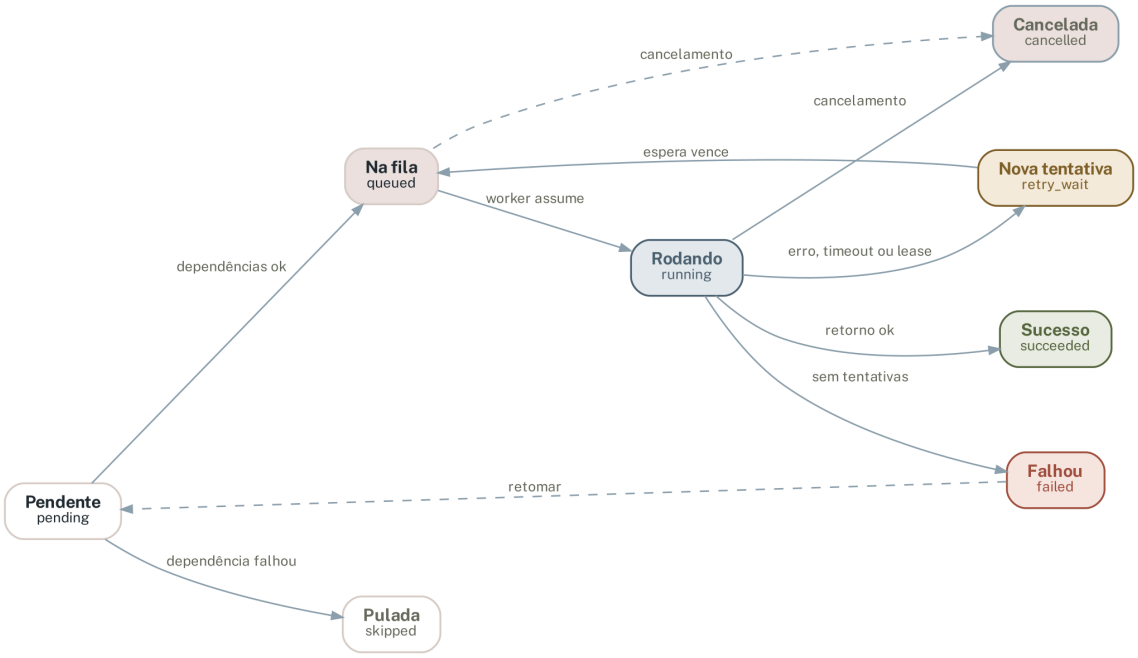



Processos do Delco e por onde passa cada informação

## Processos

| Processo | Comando | Responsabilidade |
|---|---|---|
| API | `delco api` | Daphne servindo o Django: REST no Django REST Framework, WebSocket no Channels, painel pelo WhiteNoise e Swagger. Aplica as migrações ao subir e cria as contas de demonstração. |
| Agendador | `delco scheduler` (`manage.py scheduler`) | Lê a pasta de workflows a cada 30 s; a cada segundo (ou na hora, quando acordado) cria execuções agendadas, promove execuções da fila, libera tarefas prontas e fecha execuções; a cada 5 s recupera leases vencidos, republica mensagens perdidas e marca workers sem sinal; a cada 10 min limpa o histórico antigo. |
| Worker | `delco worker` (`manage.py worker`) | Lê as filas com XREADGROUP, assume cada tarefa no banco, abre um processo filho por tentativa, envia heartbeat, aplica tempo limite e cancelamento, grava logs e resultado. |
| Processo filho | automático | Importa o arquivo do workflow, chama a função, captura stdout, stderr e logging e devolve o resultado pelo pipe. |

## Ciclo de vida de uma tarefa




Estados de uma tarefa (task_instances.status)

1. A execução nasce com todas as tarefas em **pendente**.
2. Quando as dependências terminam, o agendador marca a tarefa como **na fila** (COMMIT) e só depois publica a mensagem no Redis Stream da fila.
3. Um worker lê a mensagem e tenta assumir a tarefa: trava a linha com `select_for_update()` do ORM filtrando `status='queued'` e só então soma a tentativa. Se outra mensagem igual já foi processada, a tarefa não está mais na fila e a mensagem é descartada.
4. Durante a tentativa, o worker renova `lease_expires_at` a cada heartbeat (5 s) e lê o pedido de cancelamento.
5. No fim, grava a tentativa e decide: **sucesso** (com o resultado), **nova tentativa** (com `ready_at` calculado pela espera exponencial) ou **falhou**.
6. O agendador acorda na hora (sinal no Redis), libera as tarefas seguintes e fecha a execução quando tudo terminou.

## Confiabilidade e retomada

- **MySQL é a fonte da verdade.** O Redis só transporta mensagens. Se ele for apagado, as tarefas na fila sem mensagem são republicadas pela recuperação.
- **Compare-and-set.** Toda transição é condicional: ou a linha é travada com `select_for_update()` e conferida dentro de `transaction.atomic()`, ou o `.filter(status=…).update(…)` do ORM leva o estado esperado no filtro e devolve quantas linhas mudou. Mensagem repetida, worker atrasado ou dois agendadores ao mesmo tempo não produzem execução em dobro.

- **Lease.** Se o worker cai (queda de energia, OOM, contêiner morto), o lease vence em até 30 s e a tentativa vira Worker caiu; a tarefa volta para nova tentativa. Um worker que ressuscita depois disso perde a posse: o heartbeat e o resultado dele são recusados.
- **Resultados gravados.** O que terminou não roda de novo. Num fluxo de 50 passos que cai no passo 23, só o 23 é refeito (há um teste automatizado exatamente para isso).
- **Mensagens pendentes.** Se o worker morreu entre ler a mensagem e assumir a tarefa, outro worker a reivindica com XAUTOCLAIM.
- **Vários agendadores.** As linhas são travadas com `select_for_update(skip_locked=True)` (`FOR UPDATE SKIP LOCKED` no MySQL 8) e a chave única por horário (índice funcional em `workflow_id` e `logical_date` das execuções agendadas) impede duas execuções agendadas para o mesmo horário.
- **Desligamento gracioso.** Ao receber SIGTERM o worker para de pegar tarefas e espera as que estão rodando (30 s). O que não terminar volta para a fila.
- **Isolamento.** Cada tentativa é um processo filho em grupo próprio. Travar, estourar memória ou chamar `os._exit` derruba só a tarefa.

## Filas e paralelismo

Cada fila é um Redis Stream (`delco:queue:<nome>`) com o grupo de consumidores `workers`. Um worker consome as filas de `DELCO_WORKER_QUEUES` e pede ao Redis só a quantidade de mensagens que cabe nas vagas livres. Para dar prioridade a tarefas críticas, suba workers dedicados: `delco worker --queues critical --concurrency 2`. O paralelismo total é a soma das vagas dos workers online; `max_active_runs` limita quantas execuções do mesmo workflow rodam juntas.

## Agenda

A agenda usa cron de 5 campos no fuso do workflow. O agendador guarda a próxima data em `workflows.next_run_at`; ao chegar a hora cria a execução com `logical_date` igual àquele horário e calcula a seguinte. Não há recuperação de horários perdidos: se o agendador ficar parado por três dias, ele roda uma vez e segue a agenda. Pausar um workflow impede novas execuções agendadas; ao retomar, a agenda recomeça a partir de agora.

## Tempo real

Workers e agendador publicam eventos no canal `delco:events` do Redis (execução, tarefa, worker, log). Cada processo da API mantém uma assinatura e repassa o evento aos grupos do Django Channels: `delco.eventos` (todos os navegadores) ou `delco.tentativa.<id>` (só quem está com aquela tentativa aberta). Os eventos são um aviso: o painel sempre busca o estado oficial na API.

## Django por dentro

| Peça | Onde | O que faz |
|---|---|---|
| Modelos | `delco/models.py` | As 10 tabelas, com CHECK, índices, chave única por horário (índice funcional) e `ON DELETE` no próprio banco (`DB_CASCADE` e `DB_SET_NULL`, do Django 6.1). |

| Peça | Onde | O que faz |
|---|---|---|
| Migrações | `delco/migrations/` | Geradas pelo `makemigrations`; o atalho `delco migrate` aplica com trava nomeada do MySQL (`GET_LOCK`), para vários processos subirem juntos. |
| Transições | `delco/store.py` | Todas as mudanças de estado com o ORM, `transaction.atomic()` e `select_for_update`. |
| Consultas do painel | `delco/api/queries.py` | Agregações com `Count(filter=…)`, janelas (`RowNumber`), `TruncHour` e subconsultas. |
| API | `delco/api/views.py`, `serializers.py`, `auth.py`, `errors.py` | Views do DRF, validação com serializers, autenticação por token, permissões por papel e erro num formato único. |
| WebSocket | `delco/api/consumers.py`, `events.py` | Consumer do Channels autenticado pelo token e ponte Redis → grupos. |
| Processos | `delco/management/commands/` | `worker`, `scheduler`, `seed` e `createuser` como comandos do Django. |
| Configuração | `delco/settings.py`, `urls.py`, `asgi.py` | Tudo das variáveis `DELCO_*`; MySQL pelo mysqlclient; CSP; WhiteNoise; rotas e ASGI para o Daphne. |

**Conexões.** Cada processo abre a própria conexão com o MySQL: o worker usa uma por vaga e a fecha ao fim de cada tentativa; o agendador fecha a cada ciclo. Assim, quedas do MySQL se resolvem sozinhas: a conexão quebrada é descartada e a próxima volta nova. A sessão roda em UTC (`SET time_zone = '+00:00'`) e em READ COMMITTED.

## Estrutura de pastas

```
Workflow-Orchestration-Engine/
├── delco/
│   ├── sdk.py           Workflow, Task e validação do grafo
│   ├── context.py       objeto ctx entregue às tarefas
│   ├── helpers.py       shell() e http_json()
│   ├── loader.py        importa os arquivos de workflow
│   ├── schedule.py      cron, fuso, espera exponencial e descrição da agenda
│   ├── models.py        modelos do Django (as 10 tabelas)
│   ├── migrations/      migrações do Django
│   ├── store.py         todas as transições de estado (ORM)
│   ├── bus.py           Redis: filas, eventos e sinal de despertar
│   ├── scheduler.py     agendador e recuperação
│   ├── worker.py        worker pool, heartbeat, timeout e cancelamento
│   ├── runner.py        código do processo filho
│   ├── accounts.py      usuários, sessões e tokens
│   ├── seed.py          dados de demonstração
│   ├── api/             DRF (views, serializers, permissões), Channels e consultas
│   ├── management/      comandos worker, scheduler, seed e createuser
│   ├── settings.py      configuração do Django
│   ├── urls.py · asgi.py  rotas e entrada ASGI (Daphne)
│   └── cli.py           atalhos e cliente da API
├── manage.py            utilitário do Django
├── web/                 painel (index.html, css/, js/, fonts/)
├── workflows/           exemplos (pasta montada nos contêineres)
├── tests/               pytest e workflows de teste
├── docs/                PDFs e imagens
├── scripts/criar-env.ps1
├── Dockerfile · docker-compose.yml · iniciar.cmd · iniciar.sh
└── .github/workflows/   ci.yml e diario.yml
```

Capítulo 7

# Linha de comando

## Comandos de servidor

São comandos do Django. O `delco` é um atalho para eles; os dois jeitos funcionam.

| Atalho | Comando do Django | O que faz |
|---|---|---|
| `delco api [--port 8000]` | `manage.py migrate` + `daphne delco.asgi:application` | Sobe API, WebSocket e painel (migra o banco ao iniciar). |
| `delco scheduler` | `manage.py scheduler` | Sobe o agendador. |
| `delco worker [--queues a,b] [--concurrency N] [--name X]` | `manage.py worker …` | Sobe um worker. |
| `delco migrate` | `manage.py migrate` | Cria ou atualiza as tabelas (o atalho usa trava nomeada). |
| `delco seed` | `manage.py seed` | Cria contas e uma semana de histórico de demonstração num banco vazio. |
| `delco validate [pasta]` | — | Confere os arquivos de workflow sem rodar nada. |
| `delco users create --email --name --role [--password]` | `manage.py createuser …` | Cria um usuário (a senha é pedida no terminal se omitida). |

## Comandos de cliente

Falam com a API. A URL vem de `--url`, `DELCO_URL` ou do login salvo; o token, de `DELCO_TOKEN` ou do login salvo em `~/.config/delco/cli.json` (permissão 600).

| Comando | O que faz |
|---|---|
| `delco login --email E [--days 90]` | Entra, cria um token pessoal para a CLI e o salva. |
| `delco logout` | Revoga e apaga o token salvo. |
| `delco workflows` | Lista workflows, estado, agenda, próxima execução e últimas execuções. |
| `delco trigger NOME [-p chave=valor] [--params-json '{}'] [--note] [--wait]` | Dispara. Com `--wait` acompanha até o fim e sai com código 1 se falhar (bom para CI). |
| `delco runs [--workflow] [--status] [--limit]` | Lista execuções. |
| `delco run ID` | Mostra uma execução e suas tarefas. |
| `delco logs ID TAREFA [--attempt N]` | Imprime o log (e o traceback) de uma tentativa. |
| `delco cancel ID` | Cancela. |

| Comando | O que faz |
| --- | --- |
| `delco retry ID [--from TAREFA] [--wait]` | Retoma da falha ou a partir de uma tarefa. |
| `delco pause NOME` · `delco resume NOME` | Pausa ou retoma a agenda. |
| `delco workers` | Lista workers, filas e vagas ocupadas. |

```
$ delco trigger relatorio_vendas_diario -p pasta_saida=/tmp/rel --wait
Execução #131 criada para relatorio_vendas_diario.
  rodando    0/11 concluídas · 1 rodando · 0 com falha
  rodando    3/11 concluídas · 5 rodando · 0 com falha
  sucesso    11/11 concluídas · 0 rodando · 0 com falha
```

Capítulo 8

# API

## Primeiros passos

A API é feita com o Django REST Framework. A documentação interativa (Swagger) fica em `/api/docs` e o esquema OpenAPI 3, gerado pelo drf-spectacular, em `/api/openapi.json`. Todas as rotas respondem JSON e, exceto login e saúde, exigem `Authorization: Bearer <token>` (botão Authorize no Swagger).

```
TOKEN=$(curl -s -X POST localhost:8080/api/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"operador@delco.dev","password":"Delco@2026"}' | jq -r .token)

curl -s -X POST localhost:8080/api/workflows/ola_delco/runs \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"params": {"nome": "Delco"}, "note": "teste pela API", "source": "api"}'
# {"id": 132, "status": "queued", "workflow": "ola_delco"}
```

## Rotas

| Método e rota | Papel | Descrição |
|---|---|---|
| `GET /api/health` | livre | Saúde do banco e do Redis (503 se o banco cair). |
| `POST /api/auth/login` | livre | Devolve token de sessão, validade e usuário. |
| `POST /api/auth/logout` · `GET /api/auth/me` | leitura | Encerra a sessão · dados do usuário. |
| `PUT /api/auth/me/password` | leitura | Troca a senha e encerra as outras sessões. |
| `GET · POST /api/tokens` · `DELETE /api/tokens/{id}` | leitura | Tokens pessoais. |
| `GET /api/workflows` · `GET /api/workflows/{nome}` | leitura | Lista e detalhe com grafo e números de 30 dias. |
| `GET /api/workflows/{nome}/grid?limit=20` | leitura | Matriz execuções × tarefas. |
| `POST /api/workflows/{nome}/runs` | operador | Dispara com `params`, `note` e `source`. |
| `POST /api/workflows/{nome}/pause` · `/resume` | operador | Pausa ou retoma a agenda. |
| `GET /api/load-errors` | leitura | Arquivos de workflow com erro. |
| `GET /api/runs?workflow=&status=&page=&size=` | leitura | Execuções paginadas. |
| `GET /api/runs/{id}` | leitura | Execução com tarefas, grafo e parâmetros. |
| `POST /api/runs/{id}/cancel` | operador | Cancela (409 se já terminou). |
| `POST /api/runs/{id}/retry` | operador | Retoma; corpo opcional `{"from_task": "nome"}`. |
| `GET /api/runs/{id}/tasks/{nome}` | leitura | Tarefa com resultado e todas as tentativas. |
| `GET /api/attempts/{id}/logs?after=&limit=` | leitura | Linhas de log a partir de um id. |

| Método e rota | Papel | Descrição |
|---|---|---|
| `GET /api/overview` · `/api/workers` · `/api/queues` | leitura | Indicadores, workers e filas. |
| `GET · POST /api/users` · `PATCH /api/users/{id}` | admin | Usuários. |
| `GET /api/audit` | admin | Auditoria paginada. |

## Erros

Todo erro tem o mesmo formato:

```
{ "code": "validation_error", "message": "Revise os campos destacados.",
  "fields": { "password": "A senha precisa ter pelo menos 8 caracteres." } }
```

| Status | Quando |
|---|---|
| 401 | Sem token, token vencido ou revogado, senha errada |
| 403 | Papel sem permissão |
| 404 | Workflow, execução, tarefa ou tentativa inexistente |
| 409 | Ação impossível no estado atual (cancelar o que terminou, retomar o que está rodando) |
| 422 | Dados inválidos (detalhe em `fields`) |
| 423 | Conta bloqueada por tentativas erradas |

## WebSocket

Conecte em `/api/ws?token=…`. O servidor envia `{"type":"hello"}` e depois os eventos. Para receber o log de uma tentativa, envie `{"action":"logs","attempt_id":42}` (e `{"action":"unlogs"}` para parar).

| Evento | Campos |
|---|---|
| `run` | `id`, `status` |
| `task` | `id`, `run_id`, `name`, `status`, `attempt`, `progress` |
| `worker` | `id`, `status` (online, stopping, offline) |
| `log` | `attempt_id`, `task_id`, `run_id`, `lines` [id, at, level, message] |
| `workflows` | `changed`: workflows com nova versão, pausados ou retomados |

Capítulo 9

# Configuração

## Variáveis

No Docker, edite o `.env` (modelo em `.env.example`) e rode `docker compose up -d`. Fora dele, exporte as variáveis antes de subir cada processo.

| Variável | Padrão | Para que serve |
|---|---|---|
| `DELCO_PORT` | 8080 | Porta do painel e da API no computador (Docker). |
| `DELCO_DB_USER` · `_PASSWORD` · `_NAME` · `_PORT` | delco · gerada · delco · 3306 | MySQL do Docker Compose. Use senha só com letras e números. |
| `DELCO_DATABASE_URL` | montada pelo Compose | URL do MySQL (fora do Docker). |
| `DELCO_REDIS_URL` | redis://redis:6379/0 | URL do Redis. |
| `DELCO_SEED` | true | Contas e histórico de demonstração em banco vazio. |
| `DELCO_TIMEZONE` | America/Sao_Paulo | Fuso padrão. |
| `DELCO_WORKFLOWS_DIR` | /app/workflows | Pasta dos arquivos de workflow. |
| `DELCO_WORKER_QUEUES` | default,critical | Filas que o worker consome. |
| `DELCO_WORKER_CONCURRENCY` | 4 | Tentativas simultâneas por worker. |
| `DELCO_WORKER_NAME` | host-pid | Nome do worker no painel. |
| `DELCO_WORKER_GRACE_SECONDS` | 30 | Espera pelas tarefas ao desligar. |
| `DELCO_HEARTBEAT_SECONDS` | 5 | Intervalo do heartbeat. |
| `DELCO_LEASE_SECONDS` | 30 | Tempo sem heartbeat até a tentativa ser dada como perdida. |
| `DELCO_REDELIVER_AFTER_SECONDS` | 60 | Idade para checar mensagens perdidas e reivindicar pendentes. |
| `DELCO_TICK_SECONDS` · `_SYNC_SECONDS` · `_RECOVERY_SECONDS` | 1 · 30 · 5 | Ritmo do agendador. |
| `DELCO_RETENTION_DAYS` | 30 | Execuções terminadas há mais tempo são apagadas. |
| `DELCO_MAX_OUTPUT_BYTES` | 1048576 | Tamanho máximo do resultado de uma tarefa. |
| `DELCO_MAX_LOG_LINES` | 5000 | Linhas de log guardadas por tentativa. |
| `DELCO_SESSION_HOURS` | 12 | Validade da sessão do painel. |

| Variável | Padrão | Para que serve |
|---|---|---|
| `DELCO_LOGIN_MAX_FAILURES` · `_LOCK_MINUTES` | 5 · 5 | Bloqueio por senha errada. |
| `DELCO_LOG_LEVEL` | INFO | Nível dos logs dos processos. |
| `DELCO_ALLOWED_HOSTS` | * | `ALLOWED_HOSTS` do Django (separe por vírgula). |
| `DELCO_SECRET_KEY` | sorteada | `SECRET_KEY` do Django; o Delco não assina cookies nem sessões com ela. |
| `DJANGO_SETTINGS_MODULE` | delco.settings | Módulo de configuração (já definido no Docker e no `manage.py`). |

## Checklist para produção

- `DELCO_SEED=false` num banco vazio e um administrador criado pela CLI.
- Senha forte do MySQL no `.env` e o arquivo fora do Git (já está no `.gitignore`).
- HTTPS na frente da API (proxy reverso como Nginx, Caddy ou Traefik), repassando também o WebSocket `/api/ws`.
- `DELCO_ALLOWED_HOSTS` com o domínio do painel (por exemplo `delco.suaempresa.com`).
- Não expor as portas do MySQL e do Redis na internet (o Compose publica o MySQL só em 127.0.0.1).
- Backups do MySQL agendados (veja o manual do banco). O Redis não precisa de backup.
- Workers suficientes: some as vagas e compare com o pico de tarefas simultâneas na tela Workers.
- `DELCO_RETENTION_DAYS` de acordo com quanto histórico você quer manter.

Capítulo 10

# Testes e automação

## Pytest

A suíte usa MySQL e Redis reais e sobe agendadores e workers de verdade dentro do processo de teste.

```
docker run -d --name delco-mysql-teste -p 127.0.0.1:3307:3306 -e MYSQL_ROOT_PASSWORD=delco \
  -e MYSQL_DATABASE=delco_test -e MYSQL_USER=delco -e MYSQL_PASSWORD=delco mysql:8.4
docker run -d --name delco-redis-teste -p 127.0.0.1:6380:6379 redis:7-alpine
pip install -r requirements-dev.txt && pip install -e .
export DELCO_TEST_DATABASE_URL=mysql://delco:delco@127.0.0.1:3307/delco_test
export DELCO_TEST_REDIS_URL=redis://127.0.0.1:6380/15
python -m pytest
python manage.py makemigrations --check   # modelos e migrações em dia
```

| Arquivo | O que cobre |
|---|---|
| `test_unit.py` | Montagem e validação do grafo, cron e fuso, descrição da agenda, espera exponencial, regras de disparo, senhas, carregador de arquivos, utilitários e CLI. |
| `test_engine.py` | Fluxo linear, paralelismo, retentativas, falha definitiva com tarefas puladas, all_done e one_failed, retomada da falha e a partir de uma tarefa, tempo limite, processo que morre, resultado grande demais e cancelamento. |
| `test_reliability.py` | Queda do worker no passo 23 de 50 com retomada só daquele passo, mensagem repetida, mensagem perdida, Redis apagado, lease vencido, max_active_runs, dois agendadores no mesmo horário, pausa, worker sem sinal, desligamento gracioso e limpeza. |
| `test_api.py` | Login e bloqueio, papéis, workflows, execuções, logs, tokens, senha, usuários, auditoria, formato de erro, WebSocket (Channels) e OpenAPI. As requisições passam por toda a pilha do Django. |
| `test_cli.py` | Fluxo completo pela CLI contra a API, validate, migrate e users. |

São 77 testes, com cobertura acima de 90% (o mínimo exigido é 80%).

## Integração contínua

`.github/workflows/ci.yml` roda a cada envio, em pull requests e uma vez por dia: valida os workflows de exemplo, roda `manage.py check` e `makemigrations --check`, executa a suíte com MySQL 8.4 e Redis como serviços do GitHub Actions e monta a imagem Docker.

## Atividade diária

`.github/workflows/diario.yml` registra duas vezes por dia, em horários fixos, uma linha em `.github/atividade.txt` (mantém só as últimas 60) e faz o commit com o nome e o e-mail definidos nos secrets `GIT_NAME` e `GIT_EMAIL` do repositório. Cada turno tem um horário reserva, e a rotina não registra duas vezes o mesmo turno. Envios que só mudam esse arquivo não disparam o CI.

Capítulo 11

# Solução de problemas

| Sintoma | Causa provável e solução |
|---|---|
| O navegador não abre ou mostra erro de conexão | Os contêineres ainda estão subindo. Veja `docker compose ps` e `docker compose logs -f api`. |
| port is already allocated | Outra aplicação usa a porta. Troque `DELCO_PORT` (ou `DELCO_DB_PORT`) no `.env`. |
| Workflow novo não aparece | Aguarde até 30 s. Se continuar sumido, veja o aviso de arquivos com erro na tela Workflows ou rode `delco validate`. |
| Execução fica em Na fila | Outra execução do mesmo workflow está rodando e `max_active_runs` é 1, ou o agendador está parado. |
| Tarefas ficam em Na fila sem rodar | Nenhum worker online consome aquela fila. Confira a tela Workers e `DELCO_WORKER_QUEUES`. |
| Tentativa marcada como Worker caiu | O worker parou de enviar heartbeat (queda, falta de memória, reinício). A tarefa volta sozinha para nova tentativa. |
| Tarefa falha com Tempo limite atingido | Aumente `timeout` na tarefa ou use `timeout=None`. |
| O resultado tem N bytes | O retorno passou de 1 MB. Grave os dados fora e devolva só a referência. |
| A tarefa não existe mais | O arquivo mudou durante a execução. Ajuste e use Retomar da falha. |
| Painel mostra Reconectando… | O WebSocket caiu (proxy sem suporte a WebSocket ou API reiniciando). Ele volta sozinho; atrás de proxy, habilite o upgrade em `/api/ws`. |
| Conta bloqueada | Espere 5 minutos ou peça a um administrador para desbloquear em Usuários. |
| Bad Request (400) ao abrir pelo domínio | O domínio não está em `DELCO_ALLOWED_HOSTS`. Acrescente-o no `.env` e suba de novo. |
| Esqueci a senha do administrador | `docker compose exec api python manage.py createuser --role admin ...` cria outro administrador; depois troque a senha da conta antiga. |